RISK REVIEW · AI GOVERNANCE

Open-Weight LLM Risk Review

What an organisation risks when it runs a large language model downloaded from the internet without a control layer around it, and which controls a harness provides.

November 2026 Audience: management, security, data protection, IT

CONTENTS

1 Summary
2 What a harness is
3 The control layers
4 Weak defaults to avoid
5 Risk register
6 Context provenance and decision audit
7 Model origin: Chinese open-weight models
8 Regulatory notes
9 Minimum control baseline
10 Sources

1

Summary

A language model produces text. Risk appears the moment that text is connected to something that acts: a shell, a file system, a database, a mailbox or the internet. The software that sits between the model and those systems is called a harness. It decides what the model may do, records what it did, and limits the damage when the model is wrong or has been manipulated. A model downloaded from the internet comes without that layer.

The model proposes, the harness decides

Every action passes a policy check that runs in code the organisation controls. Unknown actions are refused or sent to a person, never allowed by default.

Prompt injection is not solved

No current model reliably ignores instructions hidden in emails, web pages or documents. Limiting what the model can reach is the main defence.

Audit what the model saw

Every model call should leave a record of the exact context, where each piece came from and how far it can be trusted. Without it, a decision cannot be explained or defended.

Treat downloaded weights as untrusted code

Model files can execute code when loaded, and model behaviour can contain hidden triggers. Origin and integrity must be checked before use.

Origin matters, controls matter more

Published tests show some Chinese open-weight models are easier to hijack. The same harness applies to every model; a weaker model needs stricter settings.

2

What a harness is

Harness engineering is the practice of building the environment around a model rather than the model itself. A public tutorial that rebuilds a commercial coding agent from scratch (Khan, 2026) describes four principles: the model makes decisions, tools are the only way it touches the world, context is a managed resource, and permissions are set in configuration rather than scattered through code.

For risk management the important consequence is a separation of duties. The model can only ask for an action. Whether that action runs, in what environment, with which rights, and how it is recorded, is decided by code that the organisation owns and can audit.

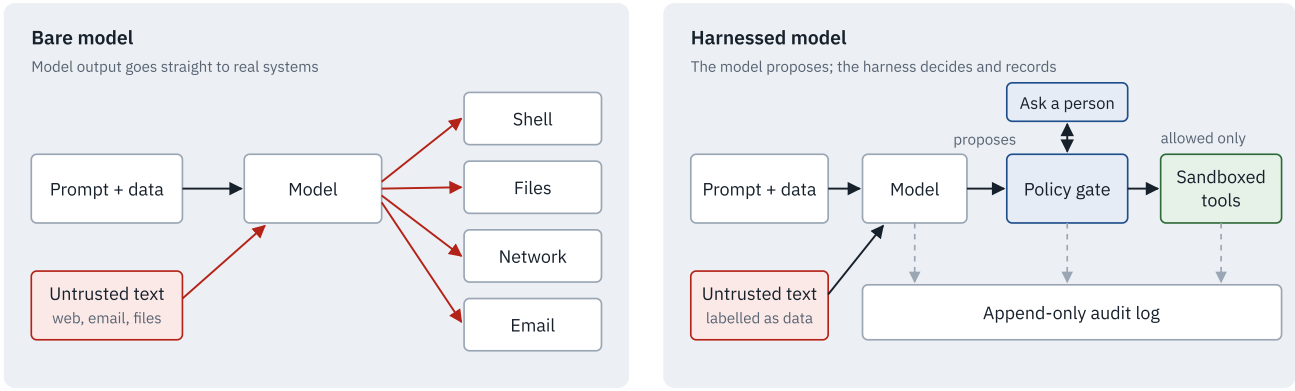


Figure 1. Without a harness, whatever the model writes becomes an action, including actions planted by hidden instructions in untrusted text. With a harness, the model can only propose. A policy gate, a person for risky steps, a sandbox and a log sit between the proposal and the real system.

3

The control layers

The reference architecture in the tutorial groups the harness into six layers. Each layer answers a specific risk. Figure 2 shows how they work together in one cycle of the agent loop.

LAYER	COMPONENTS	RISK IT ADDRESSES
Input and trust	Session management, permission gate with deny, allow and ask rules	Governance, destructive actions
Execution	Typed tool registry, file snapshots and revert	Uncontrolled actions, no way to undo

LAYER	COMPONENTS	RISK IT ADDRESSES
Knowledge and context	Instruction files loaded on demand, context compression, memory on disk	Drift from the task, context overload, provenance loss
Observability	Event bus with hooks before and after each tool call	No audit trail, no alerting
Multi-agent isolation	Sub-agents with separate context, message protocol, separate working copies per task	Agents overwriting or escalating each other
Integration	Runtime for external tool servers (Model Context Protocol)	Extends reach, and with it the attack surface

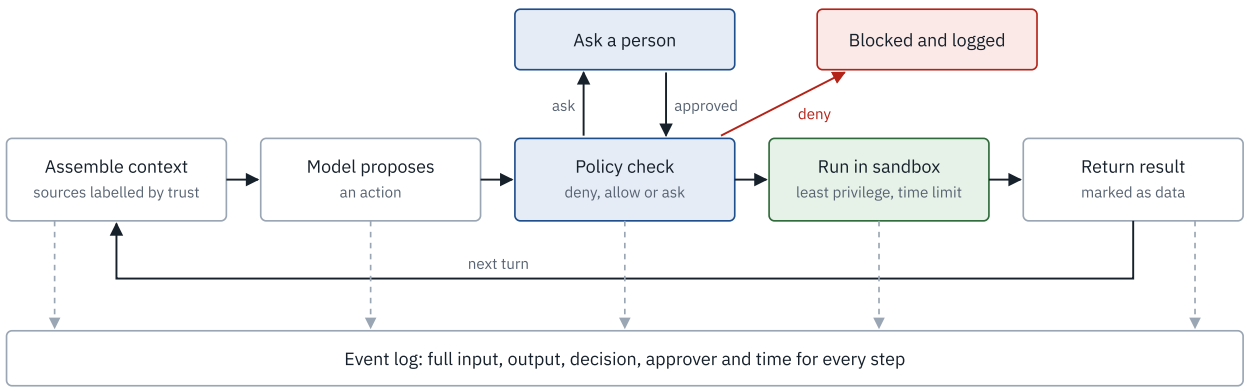


Figure 2. One cycle of a governed agent loop. The policy check is the only path from a proposal to an action. Results come back into the context marked as data, so text inside them is not treated as an instruction. Every step is written to a log the model cannot change.

4

Weak defaults to avoid

Having a harness is not enough. The tutorial's own implementation, which is written for teaching, shows how small design choices undo the protection. These are common patterns and useful test cases for any harness an organisation builds or buys.

PATTERN	WHAT GOES WRONG	BETTER DEFAULT
Allow when no rule matches	Any command nobody thought of runs without a check. In the tutorial, the code allows unmatched commands while its own diagram says they go to a person.	Deny or ask when no rule matches.
Text patterns as security rules	`ls && rm -rf data/` matches the "always allow" rule for `ls` and runs silently. A deny rule for `rm -rf /` misses `rm -rf ~`, `find / -delete` and deletes through a scripting language.	Allow lists of exact tools and arguments, a parsed command, and a sandbox that limits damage regardless of the command.
Hooks that fail open	A security hook that crashes is logged and skipped, so the action it should have blocked goes ahead.	A failing security hook blocks the action.

PATTERN	WHAT GOES WRONG	BETTER DEFAULT
Undo that only covers some actions	Snapshots cover the file-write tool and live in memory. Deletes through the shell are not covered, and all snapshots disappear when the process stops.	Work on copies or in version control; backups outside the agent's reach.
Logs the agent can edit	The log is a local file in the agent's working folder, and inputs are cut to 60 characters.	Full inputs, written to append-only storage the agent cannot reach.
External tools trusted on sight	External tool servers are registered automatically. Their tool descriptions go straight into the model's context and can carry instructions.	An approved list of servers, with descriptions reviewed and pinned.
Self-written instructions	The agent can write a new instruction file and load it, and its memory file is reloaded every session. Injected text can persist this way.	Instruction and memory changes go through review before they take effect.
Shell commands built from model text	Commands run through a shell with the model's text as input, which the agent's own code review flags as an injection path.	Call programs with argument lists, inside a sandbox.

> The tutorial also makes claims about the internals and revenue of the commercial product it imitates, based on what it calls reverse-engineered traces. These claims are not confirmed by the vendor. The tutorial is useful as an illustration of harness patterns, not as a description of any specific product.

Risk register

Scope: a model downloaded from a public repository and connected to tools or company data. The exposure column is an indicative rating for a setup without a harness. Each organisation should rate likelihood and impact against its own use case.

#	RISK	WITHOUT A HARNESS	HARNESS CONTROL	EXPOSURE
Governance and control				
1	**Governance**	No policy on what the model may do, no named owner, no approval flow.	Policy in configuration with deny, allow and ask rules; deny by default; a named owner; policy changes under version control.	High
2	**Destructive actions**	Deleted or overwritten files, forced code pushes, dropped tables, sent emails.	Human approval for anything irreversible, backups, dry-run mode, separate test and production credentials.	High
3	**Lack of observability**	Nobody can tell what the model did, why, or with which data.	Hooks before and after every tool call, structured logs with full input and output, alerts.	High
4	**Provenance of actions**	No proof of which model, prompt or person led to a change.	Append-only logs outside the agent's reach, session IDs, signed commits, a record of who approved what.	High
5	**Excess privilege**	The agent runs with every right of the user or administrator who started it.	A separate service identity, a sandbox or container, read-only by default.	High
Security				
6	**Prompt injection**	Text in an email, web page, document or tool description becomes a command.	Tool output treated as data, least privilege, approval for every action that leaves the system.	High
7	**Data exfiltration**	The model sends data out through a download command, a web request or a URL parameter.	Network egress blocked by default, an allow list of outbound domains, secrets kept out of the context.	High
8	**Secrets exposure**	The model reads keys, tokens or configuration files and repeats them.	No secrets in the working folder, output scanning, access through a secrets vault.	High
9	**Supply chain**	Model files in the pickle format, and loaders that run code shipped with the model, can execute arbitrary code when loaded.	Safetensors format only, no remote code execution at load time, loading in an isolated environment.	High

#	RISK	WITHOUT A HARNESS	HARNESS CONTROL	EXPOSURE
10	**Memory poisoning**	Injected text is saved to memory and reloaded in every later session.	Reviewed memory writes; nothing the model reads can change its standing instructions.	Medium
	Model integrity			
11	**Model provenance**	Unknown origin, a modified re-upload, unclear licence terms.	Download from the original publisher only, verify hashes, keep a register of model versions and licences.	Medium
12	**Hidden behaviour and backdoors**	Trigger-based behaviour that standard safety training does not remove.	Cannot be fixed inside the model. Contain it with a sandbox, approval gates and tests on the organisation's own tasks.	Medium
13	**Hallucination and false completion**	The model reports success without evidence.	Verification steps such as tests and read-back before a task counts as done.	Medium
	Operations			
14	**Runaway loops and cost**	Endless loops and uncontrolled compute or token spend.	Step and time limits, budgets, a kill switch, interrupts.	Medium
15	**Agents interfering**	Parallel agents overwrite each other's work or amplify each other's mistakes.	Separate working copies, locks, human review before merging.	Medium
16	**Reproducibility and support**	No pinned versions, no vendor patches, no security advisories.	Pinned model and harness versions, scheduled re-evaluation.	Medium
	Data, context and decisions			
17	**No record of what the model saw**	After a bad decision the input cannot be reconstructed.	Full prompt stored per call with a context manifest (section 6).	High
18	**Lossy or rewritten context**	Summaries and truncation change facts without anyone noticing.	Summaries keep links to their sources; originals are archived.	Medium
19	**Untraceable sources**	A data point has no ID, version or timestamp.	Every context item carries source ID, version or hash and retrieval time.	High
20	**Mixed trust levels**	Rules, user requests, internal records and web text all look the same to the model.	Trust labels and separate channels for instructions and data.	High
21	**Stale data**	A record changed after retrieval, or a summary outlives its source.	Version checks at decision time, expiry on cached context.	Medium
22	**Access leakage through retrieval**	Search pulls in records the requesting user may not see, and the model repeats them.	Filter by the end user's rights before records enter the context.	High

#	RISK	WITHOUT A HARNESS	HARNESS CONTROL	EXPOSURE
23	**Personal data in context and logs**	Full logging creates a new store of personal data.	Retention limits, access control, minimisation, a data protection impact assessment.	Medium
24	**Unfaithful explanations**	Citations or stated reasoning do not reflect what drove the output.	Citations checked in code; tests that remove a source and re-run.	Medium
25	**Decisions that cannot be replayed**	A decision cannot be re-run to check it.	Pinned model, settings, prompt template and stored context.	Medium
	Compliance			
26	**Regulatory and contractual**	Personal data in prompts and logs, duties under the EU AI Act, model licence terms.	Data minimisation, retention rules, impact assessment, licence review (section 8).	High

6

Context provenance and decision audit

A model decides on the basis of its context: the rules, the request, the records and the documents placed in front of it for that call. For an audit, two questions matter. What did the model see? And what did it use? The first can be answered completely. The second can only be narrowed down.

Many harness features that save cost or context space also remove provenance. The tutorial shows several of them:

MECHANISM	WHAT HAPPENS TO PROVENANCE
Context compression	Older turns are replaced by a summary the model writes itself. Later decisions rest on that summary, with no link back to the original records. The summary can contain errors or injected text, and it is reloaded every session.
Sub-agents	The main agent only receives the sub-agent's final text. The file reads and searches behind it are discarded, so the main decision has no evidence trail.
Background notifications	Results from background tasks are inserted into the conversation as if the user had written them. Tool output then carries the authority of a person, and the log cannot tell the two apart.
Instruction files loaded on demand	Instructions arrive as tool results from files the agent can edit. A log line that says which file was loaded does not say which version.
Memory and task files on disk	Persistent and writable by the agent, with no record of who wrote each line, when, or why.
External tool servers	Data from outside servers enters the context without a source label or trust level.

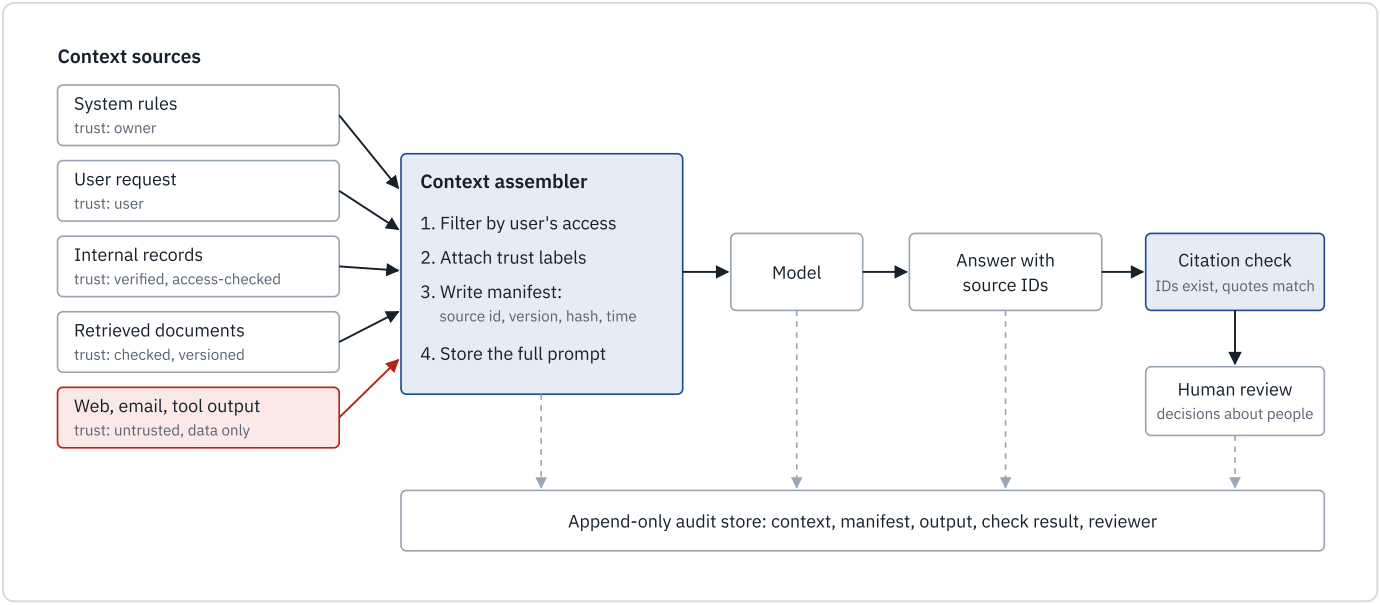


Figure 3. A context pipeline that can be audited. Every source enters with a trust level. The assembler filters by the end user's rights, records a manifest of exactly what went in, and stores the full prompt. Citations in the answer are checked against that manifest before a decision is used, and decisions about people go to a reviewer.

FULLY SOLVABLE

Record what the model saw

1. **A context manifest for every model call:** source ID, source system, version or hash, retrieval time, trust level and who could access it. Keep the full assembled prompt.
2. **Separate instructions from data:** external and tool content goes into marked data blocks. Tool output is never inserted as a user message.
3. **Traceable compression:** summaries keep links to their source records and are stored as versioned, derived items. Originals are archived, not deleted.
4. **Keep sub-agent evidence:** log each sub-agent's tool calls and sources under the parent session ID.
5. **Access checks at retrieval time:** filter records by the end user's rights before they enter the context.
6. **Tamper-evident storage:** append-only, hash-chained entries, stored outside the agent's reach, with retention rules and access control.
7. **Pin everything needed for replay:** model version, settings, prompt template version, tool versions and the manifest.

PARTLY SOLVABLE

Narrow down what the model used

1. **Require citations** to source IDs for every factual claim or decision input.
2. **Verify citations in code:** each cited ID must appear in the manifest, and quoted text must appear in that source. Flag or reject answers that fail.
3. **Leave-one-out tests** for decisions that matter: re-run without a given source and check whether the outcome changes. This shows which data actually drove the decision.
4. **Do not treat stated reasoning as an audit trail.** Research on reasoning models shows they do not always state the real basis for an answer. Use it as a hint, not as evidence.
5. **Human review for decisions about people**, with the cited sources visible to the reviewer.

7

Model origin: Chinese open-weight models

Prompt injection affects every current language model, whatever its origin. The question is whether some models are measurably easier to manipulate, and whether origin changes the data-breach picture. Published tests point to an additional risk for at least one family of Chinese models.

STUDY	FINDING	HOW TO READ IT
NIST CAISI, September 2025	Agents built on DeepSeek's most secure model tested (R1-0528) were on average 12 times more likely than the US frontier models tested to follow malicious instructions that hijacked them away from the user's task.	Hijacking is the mechanism behind data exfiltration and unwanted actions. This is the most direct evidence of added agent risk.

STUDY	FINDING	HOW TO READ IT
Cisco and University of Pennsylvania, early 2025	An automated jailbreak got harmful answers from DeepSeek R1 on all 50 sampled HarmBench prompts.	The same test gave 96% for Llama 3.1-405B and 86% for GPT-4o. Weak resistance is common among models in general, and DeepSeek R1 scored worst.
CrowdStrike, 2025	DeepSeek-R1 wrote less secure code when a prompt mentioned topics such as Tibet, Uyghurs or Falun Gong. Coverage reports up to 50% more severe vulnerabilities.	Content rules applied in training can change technical output in ways a user would not expect. Figures come from press coverage of the study.
Anthropic and others, "Sleeper Agents", 2024	Deliberately planted trigger behaviour survived standard safety training, and adversarial training sometimes hid it better.	Applies to any unverified weights. It shows that testing alone may not reveal a backdoor.

These findings concern DeepSeek models. Other Chinese model families were not covered by the studies listed here and should not be judged on these results.

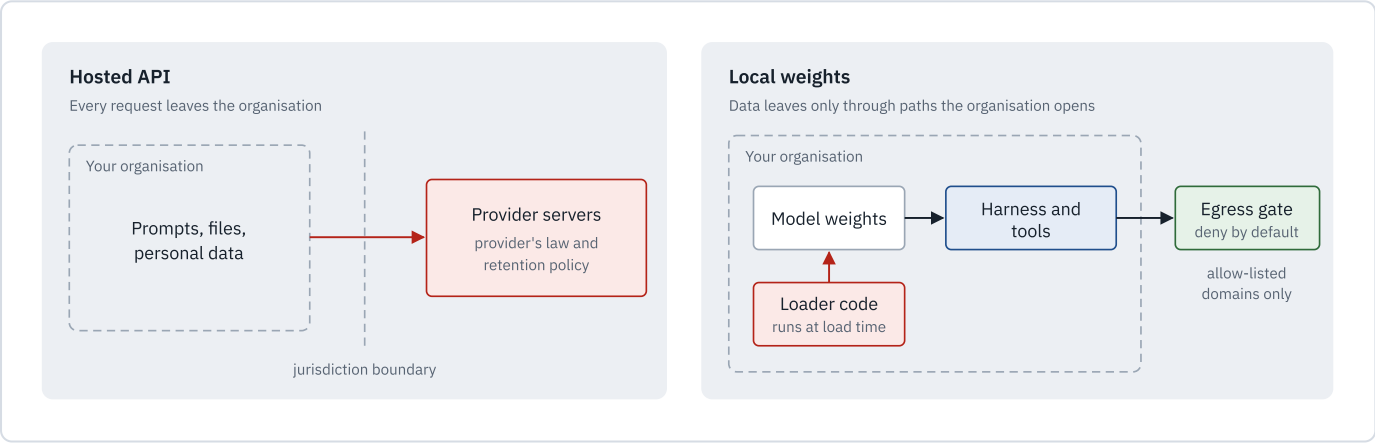


Figure 4. Hosted and local use carry different data risks. A hosted service receives every prompt under the provider's jurisdiction. Local weights cannot send data out on their own; data leaves only through tools and network paths the harness allows. The loader that reads the model file is a separate risk.

Data breach

Hosted service: prompts and data are sent to the provider and fall under the provider's jurisdiction. For a Chinese provider that is Chinese law. Check the provider's current privacy policy before any use with personal or confidential data.

Local weights: the extra risk is behavioural. A model that is easier to hijack will more readily use whatever tools and network access it has been given to move data out.

Misalignment and hidden behaviour

Planted trigger behaviour can survive safety training, and this applies to any unverified weights. For models from a jurisdiction where the state can direct companies, the question of who might plant such behaviour is more serious.

This review found no public proof of a deliberate backdoor in a Chinese model. What is documented is weaker resistance to hijacking and content rules that affect technical output.

Recommendation. A Chinese open-weight model can be acceptable for some uses, but treat it as an untrusted component with stricter harness settings: no network egress, a sandbox without secrets, human approval for every action that leaves the system, and a red-team test on the organisation's own tasks before production. Do not use the hosted service for personal or confidential data.

8

Regulatory notes

- **GDPR, Article 22.** Limits decisions based solely on automated processing that have legal or similarly significant effects on a person, and gives the person a right to human intervention and to contest the decision. That right cannot be met without knowing which data a decision used.
- **EU AI Act.** For high-risk systems, Article 10 sets requirements for data and data governance, and Article 12 requires automatic logging of events. AI used in employment and worker management is among the listed high-risk areas. Whether a given use case is high-risk depends on its purpose.
- **Model licences.** Open-weight models come with their own licence terms, which can restrict commercial use or require attribution. Record the licence with the model version.

These notes are a starting point, not legal advice. Confirm the applicable articles and their scope with legal counsel.

9

Minimum control baseline

A practical order of work. Each phase should be complete before the next starts.

PHASE 1

Before a pilot

- [ ] Named owner and written policy for what the model may do
- [ ] Model downloaded from the original publisher, hash verified, safetensors format
- [ ] No remote code execution when loading the model
- [ ] Sandbox or container with a separate service identity
- [ ] Network egress blocked by default
- [ ] No secrets or production credentials in reach

PHASE 2

Before connecting company data

- [ ] Policy gate that denies or asks when no rule matches
- [ ] Human approval for every irreversible or outbound action
- [ ] Access filtering at retrieval time
- [ ] Context manifest and full prompt stored per model call
- [ ] Append-only audit log outside the agent's reach
- [ ] Data protection impact assessment completed

PHASE 3

Before production

- [ ] Prompt injection and hijacking tests on the organisation's own tasks
- [ ] Citation checks for answers that feed decisions
- [ ] Human review for decisions about people
- [ ] Step, time and budget limits with a kill switch
- [ ] Pinned versions of model, harness, prompts and tools
- [ ] Scheduled re-evaluation and log review

10

Sources

1. Khan, F. (2026). Building Claude Code with Harness Engineering. Level Up Coding. Code: claude-code-from-scratch.
2. NIST Center for AI Standards and Innovation (2025). Evaluation of DeepSeek AI Models. See also TechRepublic coverage.
3. Cisco (2025). Evaluating Security Risk in DeepSeek and Other Frontier Reasoning Models. Comparison figures via Interesting Engineering.
4. CrowdStrike research on DeepSeek-R1 and politically sensitive prompts, as reported by VentureBeat and SC World.
5. Hubinger, E. et al. (2024). Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv:2401.05566.
6. JFrog research on malicious models on Hugging Face, as reported by Dark Reading; ReversingLabs on evasion of pickle scanning, as reported by Help Net Security.
7. Regulation (EU) 2016/679 (GDPR), Article 22. Regulation (EU) 2024/1689 (AI Act), Articles 10 and 12 and Annex III.

Where a finding is cited through press coverage, the figures should be checked against the original publication before they are quoted.